Problem 1

A manipulator with one prismatic and five rotational joints has link lengths $a_1 = 1$ and $a_2 = 0.5$ and the DH table below ($\star$ marks a joint variable).

Link	a	d	α	θ
1	a_1	0	0	$\star$
2	a_2	0	0	$\star$
3	0	$\star$	0	0
4	0	0	$\pi/2$	$\star$
5	0	0	$-\pi/2$	$\star$
6	0	0	0	$\star$

The initial pose at $t = 0\,\text{s}$ and final pose at $t = 2\,\text{s}$ are

$$ {}^0\boldsymbol{T}_6(0) = \begin{bmatrix} 1 & 0 & 0 & 1.00 \\ 0 & 0 & -1 & 0.25 \\ 0 & 1 & 0 & 0.25 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \qquad {}^0\boldsymbol{T}_6(2) = \begin{bmatrix} 0 & -1 & 0 & 0.50 \\ 1 & 0 & 0 & -0.50 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. $$

(a) Write a MATLAB function to generate the coefficients for cubic interpolation between two points given zero initial and final velocity.

(b) Write a MATLAB function to interpolate between two vectors, given cubic interpolation coefficients, an initial time, and a list of times for evaluation.

(c) For each parameterization below, generate a figure of the path in joint variable space and a figure of the Cartesian path, and play the motion as a movie using `drawArm.m`. Discuss the differences and similarities.

i. Parameter space (θ, d) from initial joint values to final.

ii. Cartesian/angle-axis space.

iii. Cartesian/quaternion space (normalize the interpolated quaternion at each sub point).

iv. Twist space.

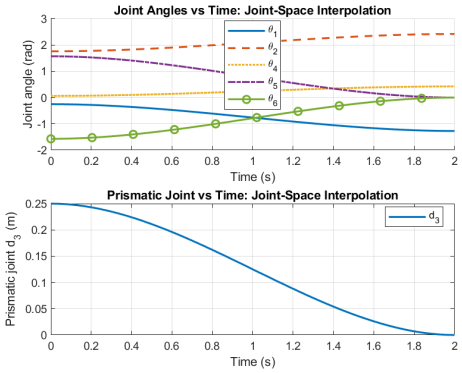
Parts (a) and (b)

`cubicCoeffs` returns the rest-to-rest cubic $q(t) = c_0 + c_1\tau + c_2\tau^2 + c_3\tau^3$, $\tau = t - t_0$, with $c_0 = q_0$, $c_1 = 0$, $c_2 = 3(q_f - q_0)/T^2$, $c_3 = -2(q_f - q_0)/T^3$, one row per element of q. `cubicInterp` evaluates these at a list of times, returning an $n \times m$ matrix. Code is in Appendix A.

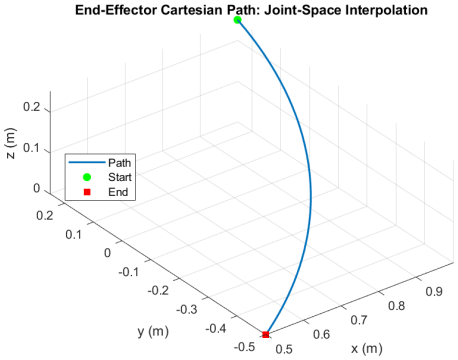
Part (c)

The inverse kinematics of the supplied closed-form solution is implemented in `ik_hw6`. The final pose has $r_{13} = r_{23} = 0$ and $r_{33} = 1$, so $\theta_5 = 0$ and the pose is a wrist singularity. In that case `ik_hw6` sets $\theta_6 = 0$ and solves $\theta_4 = \text{atan2}(r_{21}, r_{11}) - \theta_1 - \theta_2$. Each trajectory is interpolated in its own space with a rest-to-rest cubic over $0 \le t \le 2\,\text{s}$ (50 samples), converted back to a pose at every sample, and passed through the inverse kinematics. End-effector positions for the Cartesian figures come from forward kinematics (`armPositions`). All four trajectories start and end at the specified poses to machine precision.

i. Parameter space



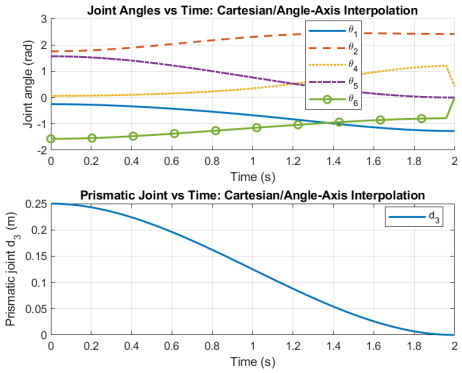
(a) Joint variables vs. time



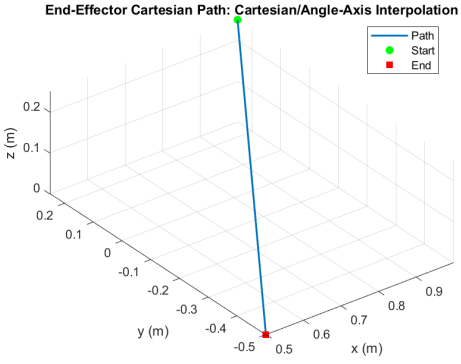
(b) Cartesian end-effector path

Figure 1: Joint-space interpolation.

ii. Cartesian/angle-axis space



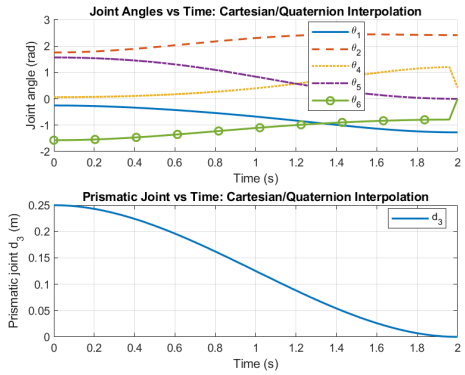
(a) Joint variables vs. time



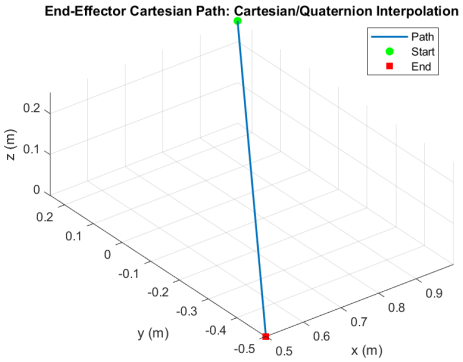
(b) Cartesian end-effector path

Figure 2: Cartesian/angle-axis interpolation.

iii. Cartesian/quaternion space



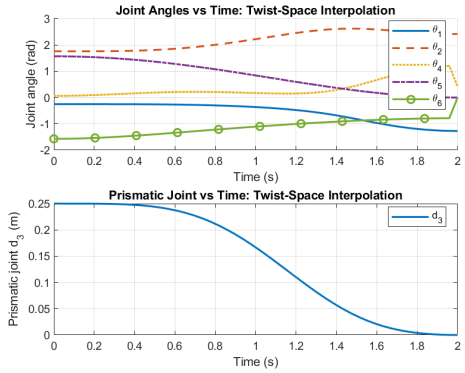
(a) Joint variables vs. time



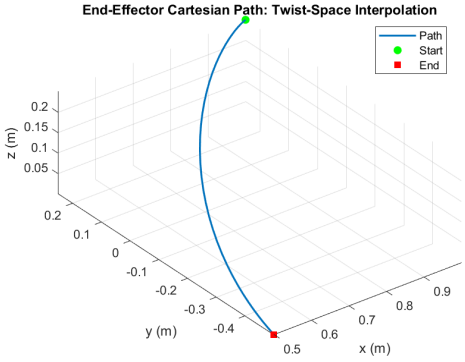
(b) Cartesian end-effector path

Figure 3: Cartesian/quaternion interpolation.

iv. Twist space



(a) Joint variables vs. time



(b) Cartesian end-effector path

Figure 4: Twist-space interpolation.

Discussion

Analyzing the four different interpolation methods, I found it interesting to note the difference in curvature over time between them. Angle-axis and quaternion both created a straight line from the start to the end pose. I believe this is because position is interpolated linearly in Cartesian space, independent of the rotation. We see that joint space and twist both create a much smoother, curved path that gradually gets to the end. However, only the joint-space method also has smooth joint curves, as is clear from the joint variables vs. time graphs I created. In parameter space, the joints have much smoother curves, implying a much smoother velocity and acceleration profile. The angle-axis, quaternion, and twist methods all show an abrupt jump in θ_4 and θ_6 in the final sample, because the final pose is a wrist singularity ($\theta_5 = 0$).

Appendix A: Code

cubicCoeffs.m (Part a)

Listing 1: cubicCoeffs.m

```
% cubicCoeffs: Cubic polynomial coefficients for a rest-to-rest trajectory.
%
% C = cubicCoeffs(q0, qf, t0, tf)
% Cubic polynomial coefficients for a rest-to-rest trajectory.
%
% C = [n x 4] coefficients per joint, q(t) = C1 + C2*tau + C3*tau^2 + C4*tau^3
%     where tau = t - t0
%
% q0 = [n x 1] initial joint values
% qf = [n x 1] final joint values
% t0 = initial time (s)
% tf = final time (s)
%
% Arael Anaya
% 10920967
% MEGN544
% 10/12/2026
function C = cubicCoeffs(q0, qf, t0, tf)
    T = tf - t0;
    dq = qf - q0;
    C = [q0, zeros(size(q0)), 3*dq/T^2, -2*dq/T^3];
```

cubicInterp.m (Part b)

Listing 2: cubicInterp.m

```
% cubicInterp: Evaluate cubic polynomial trajectories at given times.
%
% Q = cubicInterp(C, t0, tvec)
% Evaluate cubic polynomial trajectories at given times.
%
% Q = [n x m] joint values, one column per time in tvec
%
% C = [n x 4] polynomial coefficients from cubicCoeffs
% t0 = initial time the coefficients are referenced to (s)
% tvec = [1 x m] times at which to evaluate (s)
%
% Arael Anaya
% 10920967
% MEGN544
% 10/12/2026
function Q = cubicInterp(C, t0, tvec)
    tau = tvec - t0;
    Q = C(:,1) + C(:,2).*tau + C(:,3).*tau.^2 + C(:,4).*tau.^3;
```

ik_hw6.m

Listing 3: ik_hw6.m

```m
% ik_hw6: Inverse kinematics for the 1P5R arm of HW 6.
%
% thetas = ik_hw6(T, a1, a2)
% Inverse kinematics for the 1P5R arm of HW 6.
%
% thetas = [6x1] joint values [theta1; theta2; d3; theta4; theta5; theta6]
%     (rad, with d3 in length units)
%
% T = [4x4] desired end-effector pose
% a1 = length of link 1
% a2 = length of link 2
%
% Arael Anaya
% 10920967
% MEGN544
% 10/12/2026
function thetas = ik_hw6(T, a1, a2)
    px = T(1,4);  py = T(2,4);  pz = T(3,4);
    R = T(1:3,1:3);

    theta2 = 2*atan(sqrt(((a1+a2)^2 - (px^2+py^2)) / ((px^2+py^2) - (a1-a2)^2)));
    theta1 = atan2(py,px) - atan2(a2*sin(theta2), a1 + a2*cos(theta2));
    d3 = pz;
    theta5 = atan2(sqrt(R(1,3)^2 + R(2,3)^2), R(3,3));
    if abs(sin(theta5)) < 1e-9
        theta5 = 0;
        theta6 = 0;
        theta4 = atan2(R(2,1), R(1,1)) - theta1 - theta2 - theta6;
    else
        theta4 = atan2(-R(2,3)/sin(theta5), -R(1,3)/sin(theta5)) - theta1 - theta2;
        theta6 = atan2(-R(3,2)/sin(theta5), R(3,1)/sin(theta5));
    end

    thetas = [theta1; theta2; d3; theta4; theta5; theta6];
end
```

armPositions.m

Listing 4: armPositions.m

```m
% armPositions: End-effector positions along a joint-space history.
%
% P = armPositions(q, a1, a2)
% End-effector positions along a joint-space history.
%
% P = [3 x m] end-effector positions, one column per sample
%
```

```
% q = [6 x m] joint values [theta1; theta2; d3; theta4; theta5; theta6]
% a1 = length of link 1
% a2 = length of link 2
%
% Arael Anaya
% 10920967
% MEGN544
% 10/12/2026
function P = armPositions(q, a1, a2)
    m = size(q,2);
    P = zeros(3,m);
    for k = 1:m
        DH = [a1 0 0     q(1,k);
              a2 0 0     q(2,k);
              0  q(3,k) 0 0;
              0  0  pi/2  q(4,k);
              0  0 -pi/2  q(5,k);
              0  0  0     q(6,k)];
        F = fkChain(DH);
        P(:,k) = F(1:3,4,end);
    end
end
```

plotTrajectory.m

Listing 5: plotTrajectory.m

```
% plotTrajectory: Joint-space and Cartesian figures for one trajectory.
%
% plotTrajectory(t, q, P, label, figNum, saveDir, tag)
% Joint-space and Cartesian figures for one trajectory.
%
% t = [1 x m] times (s)
% q = [6 x m] joint values [theta1; theta2; d3; theta4; theta5; theta6]
% P = [3 x m] end-effector positions (m)
% label = descriptive name used in the figure titles
% figNum = figure number for the joint figure (figNum+1 is the Cartesian
    one)
% saveDir = folder to save PNGs into
% tag = short file-name tag
%
% Arael Anaya
% 10920967
% MEGN544
% 10/12/2026
function plotTrajectory(t, q, P, label, figNum, saveDir, tag)
    styles = {'-','--',':','-.','-o'};
    names = {'\theta_1','\theta_2','\theta_4','\theta_5','\theta_6'};
    rows = [1 2 4 5 6];

    f1 = figure(figNum); clf;
    subplot(2,1,1); hold on;
    for i = 1:5
```

```
        plot(t, q(rows(i),:), styles{i}, 'LineWidth', 1.5, 'MarkerIndices'
            , 1:5:numel(t));
    end
    hold off; grid on;
    legend(names, 'Location', 'best');
    xlabel('Time (s)'); ylabel('Joint angle (rad)');
    title(['Joint Angles vs Time: ' label]);
    subplot(2,1,2);
    plot(t, q(3,:), '-', 'LineWidth', 1.5); grid on;
    legend('d_3', 'Location', 'best');
    xlabel('Time (s)'); ylabel('Prismatic joint d_3 (m)');
    title(['Prismatic Joint vs Time: ' label]);
    saveas(f1, fullfile(saveDir, ['P1_joint_' tag '.png']));

    f2 = figure(figNum+1); clf;
    plot3(P(1,:), P(2,:), P(3,:), '-', 'LineWidth', 1.5); hold on;
    plot3(P(1,1), P(2,1), P(3,1), 'go', 'MarkerFaceColor', 'g');
    plot3(P(1,end), P(2,end), P(3,end), 'rs', 'MarkerFaceColor', 'r');
    hold off; grid on; axis equal;
    legend('Path', 'Start', 'End', 'Location', 'best');
    xlabel('x (m)'); ylabel('y (m)'); zlabel('z (m)');
    title(['End-Effector Cartesian Path: ' label]);
    saveas(f2, fullfile(saveDir, ['P1_cartesian_' tag '.png']));
end
```

Problem1.m (Part c)

Listing 6: Problem1.m

```
% Problem1: Trajectory planning for the 1P5R arm in joint, angle-axis,
% quaternion, and twist space
%
% Arael Anaya
% 10920967
% MEGN544
% 10/03/2026

clear; close all; clc;

addpath(genpath('../../../Functions'));

a1 = 1;
a2 = 0.5;
T06_0 = [ 1 0  0  1.00;
          0 0 -1  0.25;
          0 1  0  0.25;
          0 0  0  1   ];

T06_f = [ 0 -1  0  0.50;
          1  0  0 -0.50;
          0  0  1  0   ;
          0  0  0  1   ];

```

```
t = linspace(0,2,50);
saveDir = '../Latex/Figures';
playMovies = true;

p0 = T06_0(1:3,4);  R0 = T06_0(1:3,1:3);
pf = T06_f(1:3,4);  Rf = T06_f(1:3,1:3);
angRows = [1 2 4 5 6];      % angular joints (d3 is row 3)

% Joint space
q0 = ik_hw6(T06_0, a1, a2);
qf = ik_hw6(T06_f, a1, a2);
C = cubicCoeffs(q0, qf, 0, 2);
Q = cubicInterp(C, 0, t);
q_js = Q;

% Angle Axis
X0 = [p0; rot2AngleAxis(R0)];  Xf = [pf; rot2AngleAxis(Rf)];
Q_aa = cubicInterp(cubicCoeffs(X0, Xf, 0, 2), 0, t);
for k = 1:numel(t)
    T = [angleAxis2Rot(Q_aa(4:6,k)), Q_aa(1:3,k); 0 0 0 1];
    q_aa(:,k) = ik_hw6(T, a1, a2);
end

% Quaternion
Q0 = rot2Quat(R0);  Qf = rot2Quat(Rf);
if dot(Q0,Qf) < 0, Qf = -Qf; end
Q_q = cubicInterp(cubicCoeffs([p0; Q0], [pf; Qf], 0, 2), 0, t);
for k = 1:numel(t)
    quat = Q_q(4:7,k) / norm(Q_q(4:7,k));
    T = [quat2Rot(quat), Q_q(1:3,k); 0 0 0 1];
    q_q(:,k) = ik_hw6(T, a1, a2);
end

% Twist
xi = transform2Twist(inv(T06_0) * T06_f);
s = cubicInterp(cubicCoeffs(0, 1, 0, 2), 0, t);
for k = 1:numel(t)
    T = T06_0 * twist2Transform(s(k) * xi);
    q_tw(:,k) = ik_hw6(T, a1, a2);
end

% Unwrap angular joints so IK branch jumps of 2*pi do not appear
q_aa(angRows,:) = unwrap(q_aa(angRows,:), [], 2);
q_q(angRows,:)  = unwrap(q_q(angRows,:),  [], 2);
q_tw(angRows,:) = unwrap(q_tw(angRows,:), [], 2);

% Checks: no NaN, endpoints match, no jumps between samples
methods = {'Joint space', 'Angle-axis', 'Quaternion', 'Twist'};
hist = {q_js, q_aa, q_q, q_tw};
for i = 1:4
    qh = hist{i};
    fprintf('%-12s NaN: %d | start err %.2e | end err %.2e | max step %.3f
        \n', ...
```

```
        methods{i}, any(isnan(qh(:))), norm(qh(:,1)-q0), norm(qh(:,end)-qf), ...
        max(abs(diff(qh(angRows,:),1,2)),[],'all'));
end

% Cartesian positions from forward kinematics
P_js = armPositions(q_js, a1, a2);
P_aa = armPositions(q_aa, a1, a2);
P_q  = armPositions(q_q,  a1, a2);
P_tw = armPositions(q_tw, a1, a2);
fprintf('FK start err %.2e | end err %.2e\n', norm(P_js(:,1)-p0), norm(P_js(:,end)-pf));

% Figures
plotTrajectory(t, q_js, P_js, 'Joint-Space Interpolation',      1, saveDir, 'jointspace');
plotTrajectory(t, q_aa, P_aa, 'Cartesian/Angle-Axis Interpolation', 3, saveDir, 'angleaxis');
plotTrajectory(t, q_q,  P_q,  'Cartesian/Quaternion Interpolation', 5, saveDir, 'quaternion');
plotTrajectory(t, q_tw, P_tw, 'Twist-Space Interpolation',      7, saveDir, 'twist');

% Movies
if playMovies
    for i = 1:4
        qh = hist{i};
        figure(20+i); clf;
        for k = 1:numel(t)
            drawArm(qh(1,k), qh(2,k), qh(3,k), qh(4,k), qh(5,k), qh(6,k), 20+i, T06_f);
            title([methods{i} ' Interpolation, t = ' num2str(t(k),'%.2f') ' s']);
            drawnow; pause(0.05);
        end
    end
end
```